

Technical Stack Report: AUNTY D

1. Executive Summary

The AUNTY D platform utilizes a modern, TypeScript-first architecture organized as a monorepo. The system is bifurcated into two distinct client-side applications (Customer and Admin) supported by a centralized Node.js/Express backend. This modular approach ensures separation of concerns while maintaining type safety across the entire development lifecycle.

2. Frontend Infrastructure

The system maintains two dedicated frontends built on **React 18** and bundled with **Vite 4** for optimized development and build cycles.

2.1 Customer-Facing Application (frontend/)

The customer portal is designed for high interactivity and accessibility, utilizing a robust suite of UI tools.

- **Core Logic:** TypeScript, React 18, React Router 6.
- **Design System:** Styled with **Tailwind CSS 3**, utilizing **Radix UI** primitives for accessible components (Modals, Accordions, Dropdowns).
- **Visuals & UX:** * **Icons:** Lucide-react.
 - **Data Visualization:** Recharts for loan metrics and financial tracking.
 - **Feedback:** Sonner for toast notifications.
- **State & Form Management:** React-hook-form for efficient data entry and validation.

2.2 Admin Dashboard (admin-frontend/)

The admin portal follows a "leaner" philosophy to prioritize performance and simplicity for internal operations.

- **Styling:** Upgraded to **Tailwind CSS 4** via a PostCSS pipeline.
- **Architecture:** Avoids heavy UI libraries in favor of custom-built CSS components located in index.css.

3. Backend & Data Layer (backend/)

The server-side infrastructure is built for scalability and secure data handling.

- **Runtime Environment:** Node.js with TypeScript.
- **Web Framework:** Express 4 managing RESTful API endpoints.
- **Security:**
 - **Authentication:** JSON Web Tokens (JWT) for stateless session management.
 - **Data Privacy:** Bcryptjs for industrial-grade password hashing.
- **Storage & Communication:**
 - **File Handling:** Multer (local disk storage) for loan-related documentation.

- **Messaging:** Nodemailer via SMTP for automated email notifications.

3.1 Database Management

The system relies on a relational data model managed through modern ORM tooling.

Component	Technology
Database Engine	PostgreSQL
ORM	Prisma 6
Migration Tooling	Prisma Migrate
Schema Definition	backend/prisma/schema.prisma

4. Deployment & DevOps

While the codebase remains host-agnostic, the following infrastructure is recommended for production environments:

- **Client Hosting:** Vercel or Netlify (optimized for Vite-based React apps).
- **API Hosting:** Render or Railway (ideal for Express/Node.js runtimes).
- **Managed Database:** Neon or Supabase (PostgreSQL).

5. Version Control Baseline

As of April 2026, the primary dependencies are pinned to the following versions:

- **React:** ~18.2
- **Vite:** ~4.4
- **Express:** ~4.19
- **Prisma:** ~6.19
- **TypeScript:** 5.x

Note: For specific implementation details regarding component interaction and data flow, please refer to the internal SYSTEM_ARCHITECTURE.md documentation.